# FYP SKILLS LEARNING PLAN

Smart Academic Communication Platform - Skills

MAY 23, 2026

# Table of Contents

1. Technical Lead & Core Backend Architect ..... 4
1.1 Role ..... 4
1.2 Primary Domain ..... 4
1.3 Technologies to Use ..... 4
1.3.1 Backend ..... 4
1.3.2 Architecture ..... 4
1.3.3 Authentication & Security ..... 5
1.3.4 Database ..... 5
1.3.5 Background Processing ..... 5
1.3.6 Real-Time (if using .NET) ..... 5
1.3.7 DevOps & Collaboration ..... 5
1.3.8 Deployment ..... 6
1.4 Skills Required ..... 6
1.4.1 Core Skills ..... 6
1.4.2 Advanced Skills ..... 6
1.4.3 Soft Skills ..... 6
1.5 Main Modules ..... 6
2. Frontend Lead ..... 7
2.1 Role ..... 7
2.2 Primary Domain ..... 7
2.3 Technologies to Use ..... 7
2.3.1 Frontend ..... 7
2.3.2 State Management ..... 7
2.3.3 Routing & API ..... 7
2.3.4 UI Components ..... 8
2.3.5 Charts & Analytics ..... 8
2.3.6 Real-Time Integration ..... 8
2.3.7 Development Tools ..... 8

2.4 Skills Required ..... 8
2.4.1 Core Skills ..... 8
2.4.2 Advanced Skills ..... 8
2.4.3 UI/UX Skills ..... 9
2.5 Main Modules ..... 9
3. AI/ML Engineer ..... 9
3.1 Role ..... 9
3.2 Primary Domain ..... 9
3.3 Technologies to Use ..... 10
3.3.1 Python Ecosystem ..... 10
3.3.2 Machine Learning ..... 10
3.3.3 NLP ..... 10
3.3.4 Data Visualization ..... 10
3.3.5 Model Handling ..... 10
3.3.6 API Integration ..... 10
3.3.7 Development Tools ..... 10
3.4 Skills Required ..... 11
3.4.1 Core Skills ..... 11
3.4.2 Advanced Skills ..... 11
3.4.3 AI Engineering Skills ..... 11
3.5 Main Modules ..... 11
4. Full Stack & Real-Time Systems Engineer ..... 12
4.1 Role ..... 12
4.2 Primary Domain ..... 12
4.3 Technologies to Use ..... 12
4.3.1 Backend ..... 12
4.3.2 Real-Time ..... 12
4.3.3 Database Interaction ..... 12
4.3.4 File Handling ..... 12

4.3.5 Frontend Support ..... 12
4.3.6 Dev Tools ..... 13
4.4 Skills Required ..... 13
4.4.1 Core Skills ..... 13
4.4.2 Advanced Skills ..... 13
4.4.3 Full Stack Skills ..... 13
4.5 Main Modules ..... 13
5. Shared Technologies ..... 14
6. Recommended Git Branch Strategy ..... 14
7. Recommended Folder/Service Structure ..... 14
7.1 Backend (.NET) ..... 14
7.2 AI Service ..... 15
7.3 Frontend ..... 15
7.4 Real-Time Service ..... 15
8. Recommended Learning Focus for Each Member ..... 15
9. Final Recommendation ..... 16
9.1 Core System ..... 16
9.2 AI Layer ..... 16
9.3 Frontend ..... 16
9.4 Real-Time Layer ..... 16
9.5 Database ..... 16
10. Optional Enhancements ..... 16

**Tools, Technologies & Skills Distribution Plan**

This distribution is designed to:

- avoid overlap/confusion
- maximize each member's expertise
- maintain centralized architecture
- ensure parallel development
- keep the project academically strong

# 1. Technical Lead & Core Backend Architect

## 1.1 Role

- Talha Ahmad

## 1.2 Primary Domain

- System Design
- Core Backend
- Architecture
- Shared Services
- Security
- Integration

## 1.3 Technologies to Use

### 1.3.1 Backend

- ASP.NET Core Web API
- C#
- Entity Framework Core
- LINQ

### 1.3.2 Architecture

- Clean Architecture + VSA

- Repository Pattern
- Service Layer Pattern
- Dependency Injection
- CQRS (optional)
- RESTful API Design

### 1.3.3 Authentication & Security

- JWT Authentication
- Role-Based Authorization (RBAC)
- ASP.NET Identity
- Refresh Tokens

### 1.3.4 Database

- PostgreSQL or AzureSQL
- Database Normalization
- Indexing
- Query Optimization

### 1.3.5 Background Processing

- Hangfire
- Quartz.NET (optional)

### 1.3.6 Real-Time (if using .NET)

- SignalR

### 1.3.7 DevOps & Collaboration

- Git
- GitHub
- GitHub Projects
- Postman
- Swagger/OpenAPI

### 1.3.8 Deployment

- Docker (optional but recommended)
- IIS/Kestrel
- Nginx basics

## 1.4 Skills Required

### 1.4.1 Core Skills

- Backend architecture
- API design
- System scalability
- Authentication systems
- Database design
- Multi-service integration

### 1.4.2 Advanced Skills

- Scheduling systems
- Notification orchestration
- Rule engine development
- Performance optimization
- Logging & monitoring

### 1.4.3 Soft Skills

- Technical leadership
- Task coordination
- Integration management
- Code review

## 1.5 Main Modules

- Authentication & Authorization
- Academic Data Management

- Automation Rule Engine
- Campaign Management
- Shared Services
- Notification System
- Analytics APIs
- System Integration

# 2. Frontend Lead

## 2.1 Role

- Muhammad Raza

## 2.2 Primary Domain

- Frontend Development
- UI/UX
- Dashboard Systems
- API Integration

## 2.3 Technologies to Use

### 2.3.1 Frontend

- React.js
- JavaScript/TypeScript
- TailwindCSS

### 2.3.2 State Management

- Redux Toolkit OR Zustand
- React Context API

### 2.3.3 Routing & API

- React Router
- Axios

- Fetch API

### 2.3.4 UI Components

- ShadCN UI (optional)
- Material UI (optional)

### 2.3.5 Charts & Analytics

- Chart.js
- Recharts

### 2.3.6 Real-Time Integration

- Socket.IO Client
- SignalR Client (if using SignalR)

### 2.3.7 Development Tools

- Vite
- VS Code
- Git/GitHub
- Postman

## 2.4 Skills Required

### 2.4.1 Core Skills

- Responsive UI development
- Component-based architecture
- API integration
- State management
- Form handling

### 2.4.2 Advanced Skills

- Real-time UI updates
- Dashboard optimization
- Dynamic data visualization

- Frontend performance optimization

### 2.4.3 UI/UX Skills

- Accessibility
- Layout structuring
- Dashboard usability
- Design consistency

## 2.5 Main Modules

- Admin Dashboard
- Faculty Dashboard
- Student Panels
- Analytics UI
- Campaign Builder
- Rule Builder UI
- Chat UI
- Notification UI

# 3. AI/ML Engineer

## 3.1 Role

- Areeb Siddiqui

## 3.2 Primary Domain

- Machine Learning
- NLP
- Intelligent Analytics
- AI Integration

## 3.3 Technologies to Use

### 3.3.1 Python Ecosystem

- Python
- FastAPI OR Flask

### 3.3.2 Machine Learning

- Scikit-learn
- Pandas
- NumPy

### 3.3.3 NLP

- NLTK
- spaCy
- Transformers (optional)

### 3.3.4 Data Visualization

- Matplotlib
- Seaborn
- Jupyter Notebook

### 3.3.5 Model Handling

- Pickle/Joblib
- Model serialization

### 3.3.6 API Integration

- REST APIs
- JSON handling

### 3.3.7 Development Tools

- PyCharm/VS Code
- Git/GitHub
- Postman

## 3.4 Skills Required

### 3.4.1 Core Skills

- Data preprocessing
- Feature engineering
- Supervised learning
- NLP processing
- Classification models

### 3.4.2 Advanced Skills

- Spam filtering
- Priority prediction
- Sentiment analysis
- Recommendation systems

### 3.4.3 AI Engineering Skills

- Model evaluation
- Accuracy optimization
- Dataset preparation
- AI service deployment

## 3.5 Main Modules

- Email Classification
- Spam Detection
- Priority Prediction
- Smart Communication Analysis
- AI Recommendation Engine
- Predictive Analytics

# 4. Full Stack & Real-Time Systems Engineer

## 4.1 Role

- Furqan Moin

## 4.2 Primary Domain

- Real-Time Systems
- Supporting Backend
- Socket Communication
- File Handling
- Full-Stack Support

## 4.3 Technologies to Use

### 4.3.1 Backend

- Node.js
- Express.js

### 4.3.2 Real-Time

- Socket.IO
- WebSockets

### 4.3.3 Database Interaction

- MongoDB (optional for chat only)
- Redis (optional)

### 4.3.4 File Handling

- Multer
- Cloudinary/local storage

### 4.3.5 Frontend Support

- React.js
- TailwindCSS

### 4.3.6 Dev Tools

- Git/GitHub
- Postman
- VS Code

## 4.4 Skills Required

### 4.4.1 Core Skills

- Real-time communication
- Event-driven programming
- API development
- File upload systems

### 4.4.2 Advanced Skills

- Socket optimization
- Presence tracking
- Real-time notifications
- Media/document handling

### 4.4.3 Full Stack Skills

- React integration
- Backend/frontend coordination
- Real-time UI synchronization

## 4.5 Main Modules

- Real-Time Chat System
- Socket Communication
- File Sharing Module
- Real-Time Notifications
- Media Handling
- Chat Presence System

# 5. Shared Technologies

| *Category* | *Technologies* |
|---|---|
| *Version Control* | Git, GitHub |
| *API Testing* | Postman |
| *Documentation* | Swagger, Markdown |
| *Communication* | Discord/WhatsApp/Teams |
| *Project Management* | Trello/Jira/GitHub Projects |
| *Database Understanding* | SQL Basics |
| *Collaboration* | Pull Requests, Branching |

# 6. Recommended Git Branch Strategy

| *Branch* | *Purpose* |
|---|---|
| *main* | Stable production-ready |
| *dev* | Integration branch |
| *frontend/** | React features |
| *backend/** | .NET APIs |
| *ai/** | ML modules |
| *realtime/** | Socket/chat systems |

# 7. Recommended Folder/Service Structure

## 7.1 Backend (.NET)

```
Backend/
 ├── AuthService
 ├── AcademicService
 ├── CampaignService
 ├── NotificationService
 ├── AnalyticsService
```

```
├── Shared
```

### 7.2 AI Service

```
AI-Service/
├── models/
├── datasets/
├── training/
├── inference/
├── api/
```

### 7.3 Frontend

```
Frontend/
├── components/
├── pages/
├── services/
├── hooks/
├── layouts/
├── charts/
```

### 7.4 Real-Time Service

```
Realtime-Service/
├── sockets/
├── events/
├── chat/
├── notifications/
```

## 8. Recommended Learning Focus for Each Member

| MEMBER | SHOULD IMPROVE |
|---|---|
| **TALHA AHMAD** | Distributed systems, caching, message queues |
| **MUHAMMAD RAZA** | Advanced React patterns, charting systems |
| **AREEB SIDDIQUI** | NLP pipelines, model deployment |
| **FURQAN MOIN** | WebSocket scaling, Redis, async systems |

# 9. Final Recommendation

Our strongest possible architecture academically is:

## 9.1 Core System

- ASP.NET Core

## 9.2 AI Layer

- Python FastAPI

## 9.3 Frontend

- React + Tailwind

## 9.4 Real-Time Layer

- SignalR OR Socket.IO

## 9.5 Database

- PostgreSQL

# 10. Optional Enhancements

- Redis
- Docker
- Background queues

This gives our FYP:

- enterprise-like structure
- modern stack
- AI integration
- scalable communication system
- real-time capabilities
- strong software engineering concepts